

Graphics and Computing GPUs

Introduction to GPUs: The Core Concepts

- **Definition:** A Graphics Processing Unit (GPU) is a processor specifically optimized for 2D and 3D graphics, video, visual computing, and display
- **Architecture:** Modern GPUs are highly parallel, highly multithreaded multiprocessors optimized for visual computing
- **Visual Computing:** This represents a mix of graphics processing and computing that allows for real-time interaction with computed objects
- **Performance Scale:** GPUs provide two to three orders of magnitude more thread and data parallelism than traditional CPUs

Evolution of the GPU: From Fixed-Function to General-Purpose

- **Early Days (pre-1990s):** Graphics were handled by simple Video Graphics Array (VGA) controllers, which were essentially memory controllers connected to DRAM
- **1990s:** Incorporation of 3D acceleration functions, including triangle setup, rasterization, and texture mapping
- **Year 2000:** The term "GPU" was coined as the single-chip graphics processor incorporated nearly all details of a high-end workstation graphics pipeline
- **Modern GPUs:** Evolution into massively parallel programmable processors with hundreds of cores and thousands of threads capable of running general-purpose languages like C and C++

The Heterogeneous System: CPU vs. GPU Roles

- **Heterogeneous System:** A system that combines different processor types, such as a CPU and a GPU
- **Complementary Strengths:**
 - **CPU:** Multicore (soon manycore), primarily serial processor
 - **GPU:** Massively parallel manycore processor
- **Optimized Performance:** The best performance for many applications results from using both processors together, splitting work according to their strengths

The Real-Time Graphics Pipeline: Logical Structure and Shaders

- **Logical Graphics Pipeline:** Defines a dataflow infrastructure that maps onto GPU hardware
- **Programmable Shader Stages:**
 - **Vertex Shader:** Handles per-vertex processing, such as transforming 3D positions to screen positions
 - **Geometry Shader:** Processes primitives (lines/triangles) and can add or drop them
 - **Pixel Shader:** Performs per-fragment processing, including texturing and coloring
- **Fixed-Function Blocks:** Units like the Setup & Rasterizer and Input Assembler handle hardware-specific tasks automatically

Graphics Shading Languages: Programming the Pipeline

- **Primary Languages:** HLSL (High-Level Shading Language) and Cg (C for graphics)
- **Characteristics:**
 - **Syntax:** C-like syntax but lacks general features like pointers, file I/O, or recursion
 - **Implicit I/O:** Inputs (like texture coordinates) are often interpolated by upstream stages
 - **Independence:** Many program instances run in parallel as independent threads because they work on independent data and have no side effects

Transitioning to GPU Computing: From GPGPU to CUDA

- **GPGPU (General-Purpose computation on GPU):** An earlier approach using traditional graphics APIs and pipelines for non-graphics tasks, which was often awkward and limiting.
- **GPU Computing:** Coined for using the GPU via a parallel programming language and API without the graphics pipeline model.
- **CUDA (Compute Unified Device Architecture):** A scalable parallel programming model and platform from NVIDIA that allows programming in standard C/C++.

The CUDA Programming Model: Hierarchy of Abstraction

- **SPMD Style:** Single-Program Multiple Data software style where a programmer writes code for one thread executed by many in parallel
- **The Kernel:** A function or program for one thread designed to be executed by many threads simultaneously
- **The Hierarchy:**
 - **Threads:** The basic unit of execution
 - **Thread Blocks:** Concurrent threads that can cooperate via barrier synchronization and shared memory
 - **Grids:** A set of independent thread blocks that execute the same kernel

Data Parallel Problem Decomposition: Mapping Work to Hardware

- **Decomposition Strategy:** Partition a large result data array into blocks, and further partition each block into elements
- Graphics and Computing GPUs
 - Parallel multiprocessors compute result blocks
 - Parallel threads compute result elements
 - **Independence Requirement:** Thread blocks must be able to execute in any order (parallel or serial) to ensure scalability

Scalability & Performance Trends: The Future of GPU Computing

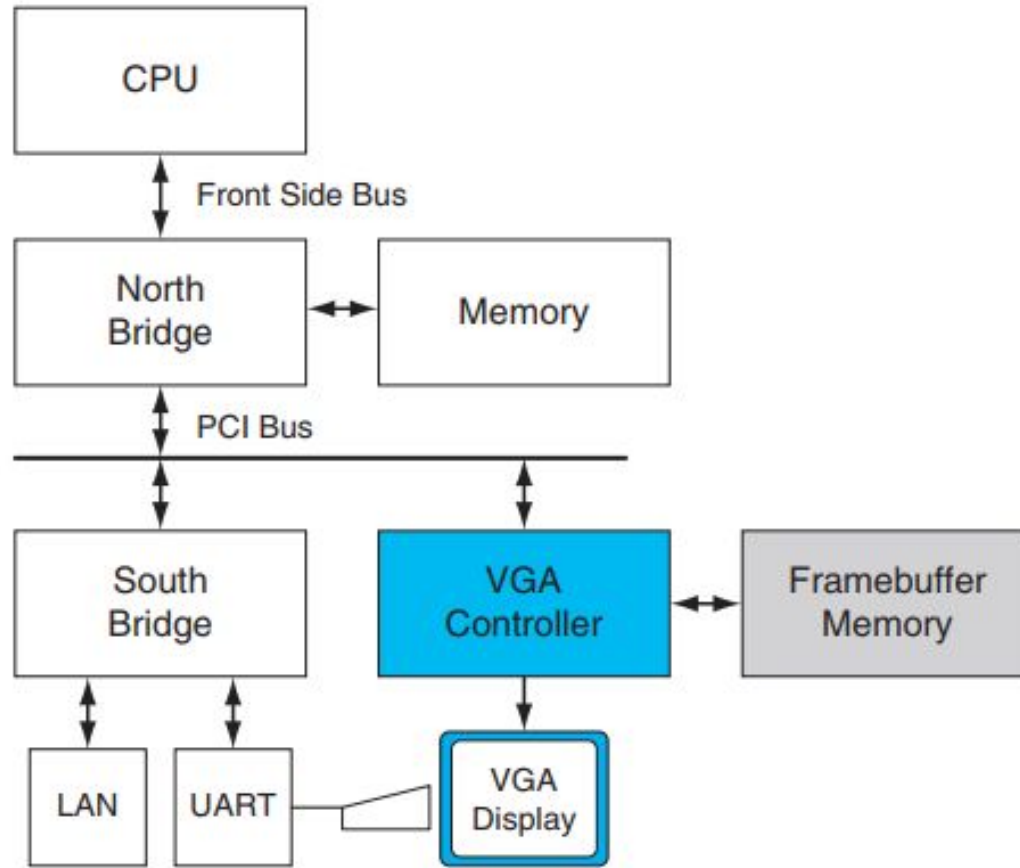
- **Moore's Law in Action:** GPU parallelism doubles approximately every 12 to 18 months
- **Transparent Scalability:** Programs are designed to automatically increase performance when run on hardware with more processors without needing recompilation
- **Precision Evolution:** Processing has moved from simple integer/fixed-point arithmetic to high-performance single and double precision floating-point

Introduction to GPU System Architectures: Framework and Scope

- **Architecture Definition:** A heterogeneous computer system using a CPU and a GPU is defined by its functional subsystems, interconnection topology, and available memory subsystems
- **Survey Scope:** Modern configurations include system services, standard programming interfaces, and internal processor architectures
- **Core Objective:** These architectures are designed to stimulate new visual computing approaches by making all applications possible through massive parallelism

The Historical PC (Circa 1990): The Legacy Foundation

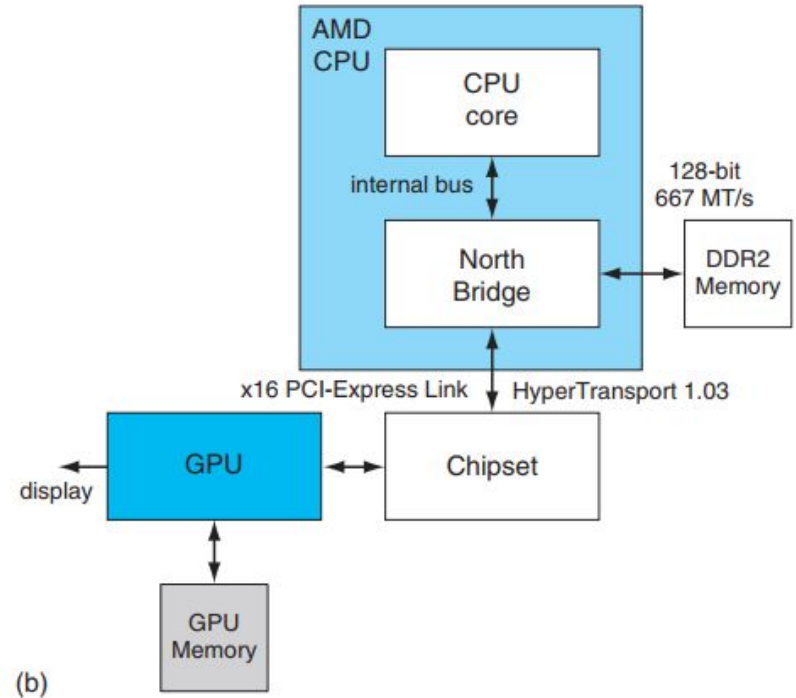
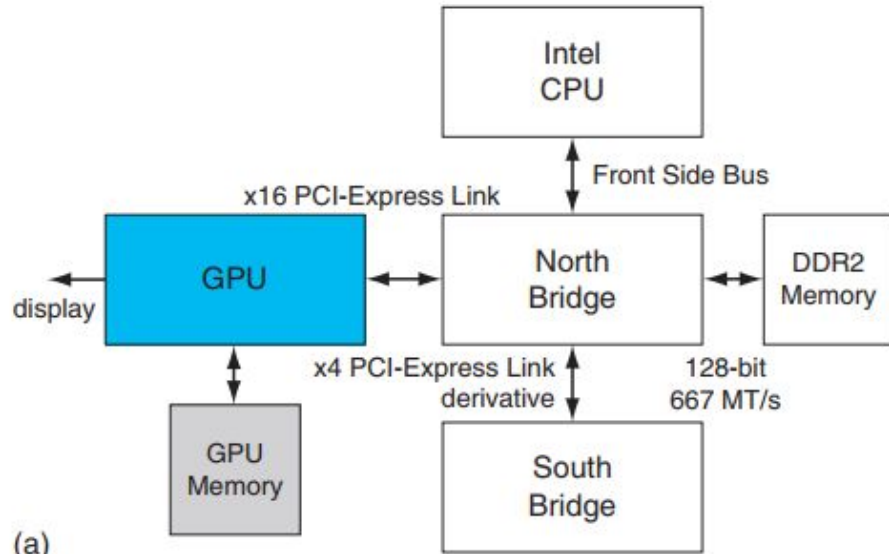
- **VGA Controller:** In legacy systems, the display was driven by a simple framebuffer subsystem known as a VGA (Video Graphics Array)
- **Interconnect:** The VGA controller was attached to the low-bandwidth PCI bus
- **Fixed Functionality:** Graphics subsystems with built-in processing elements (modern GPUs) did not exist in the PC landscape of 1990
- **Chipset Role:** High-bandwidth interfaces (CPU and memory) were managed by the North Bridge, while legacy interfaces (audio, LAN) were handled by the South Bridge



Historical PC. VGA controller drives graphics display from framebuffer memory

Contemporary Heterogeneous Systems: Discrete GPU

- **Discrete Design:** Characterized by a separate GPU and CPU, each possessing its own dedicated memory subsystem
- **High-Speed Interconnects:** Modern GPUs typically attach via a 16-lane PCI-Express (PCIe) link, providing peak transfer rates of 16 GB/s (8 GB/s in each direction)
- **Memory Access:** While both processors can access each other's memory, they do so with significantly less bandwidth than their access to directly attached "local" memory
- **Integration Trends:** In some systems, the memory controller (North Bridge) is integrated directly onto the CPU die



Contemporary PCs with (a) Intel and (b) AMD CPUs

Unified Memory Architecture (UMA): Efficiency and Trade-offs

- **Definition:** A system architecture where the CPU and GPU share a common system memory, omitting dedicated GPU memory to reduce costs
- **Performance Impact:** UMA systems generally offer lower performance because they are limited by system memory bandwidth and increased latency
- **Aperture Access:** The CPU accesses the GPU's memory range through a specific address range in the PCI-Express space

CPU and GPU Memory: Shared Access

- With CPUs and GPUs, there are multiple address spaces
- GPUs can access their own physical local memory and the CPU system's physical memory using virtual addresses that are translated by an MMU on the GPU
- The operating system kernel manages the GPU's page tables
- A system physical page can be accessed using either coherent or noncoherent PCI-Express transactions, determined by an attribute in the GPU's page table
- The CPU can access GPU's local memory through an address range (also called aperture) in the PCI-Express address space

Multi-GPU and Console Architectures: Specialized System

- **Multi-GPU Systems:** High-performance workstations and gaming rigs may use two to four GPUs in parallel, such as the NVIDIA SLI (Scalable Link Interconnect)
- **Console Design (PS3/Xbox 360):** Resemble PC architectures but use customized internal buses instead of standardized ones
- **Lifecycle:** Consoles are designed for a 5+ year lifespan with constant capability, often being reimplemented on advanced silicon processes to lower costs over time

GPU Interfaces and Drivers: Software and Hardware Interaction

- **API Coprocessing:** Applications use standard APIs like OpenGL or Direct3D to utilize the GPU as a specialized coprocessor
- **The Driver's Role:** APIs send commands, programs, and data to the GPU through a graphics device driver optimized for that specific hardware
- **Evolution of Standards:** Modern connectivity relies on PCI-Express, replacing earlier standards like AGP (Accelerated Graphics Port)

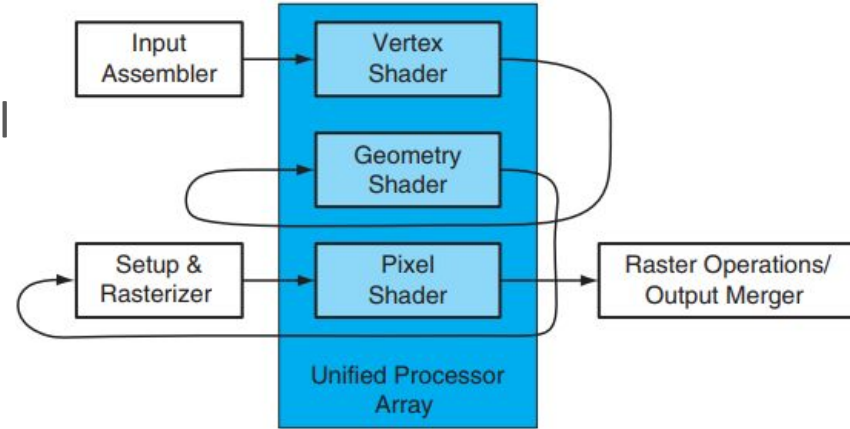
The Graphics Logical Pipeline: Dataflow and Programmability

- **Stage Overview:** The logical pipeline includes major processing stages like the Input Assembler, Rasterizer, and Output Merger
- **Programmable Stages:** The vertex, geometry, and pixel shader stages are highlighted as the primary programmable components
- **Data Recirculation:** The logical pipeline dataflow recirculates through the physical processors to complete complex frames



Unified GPU Architecture: A Converged Design Point

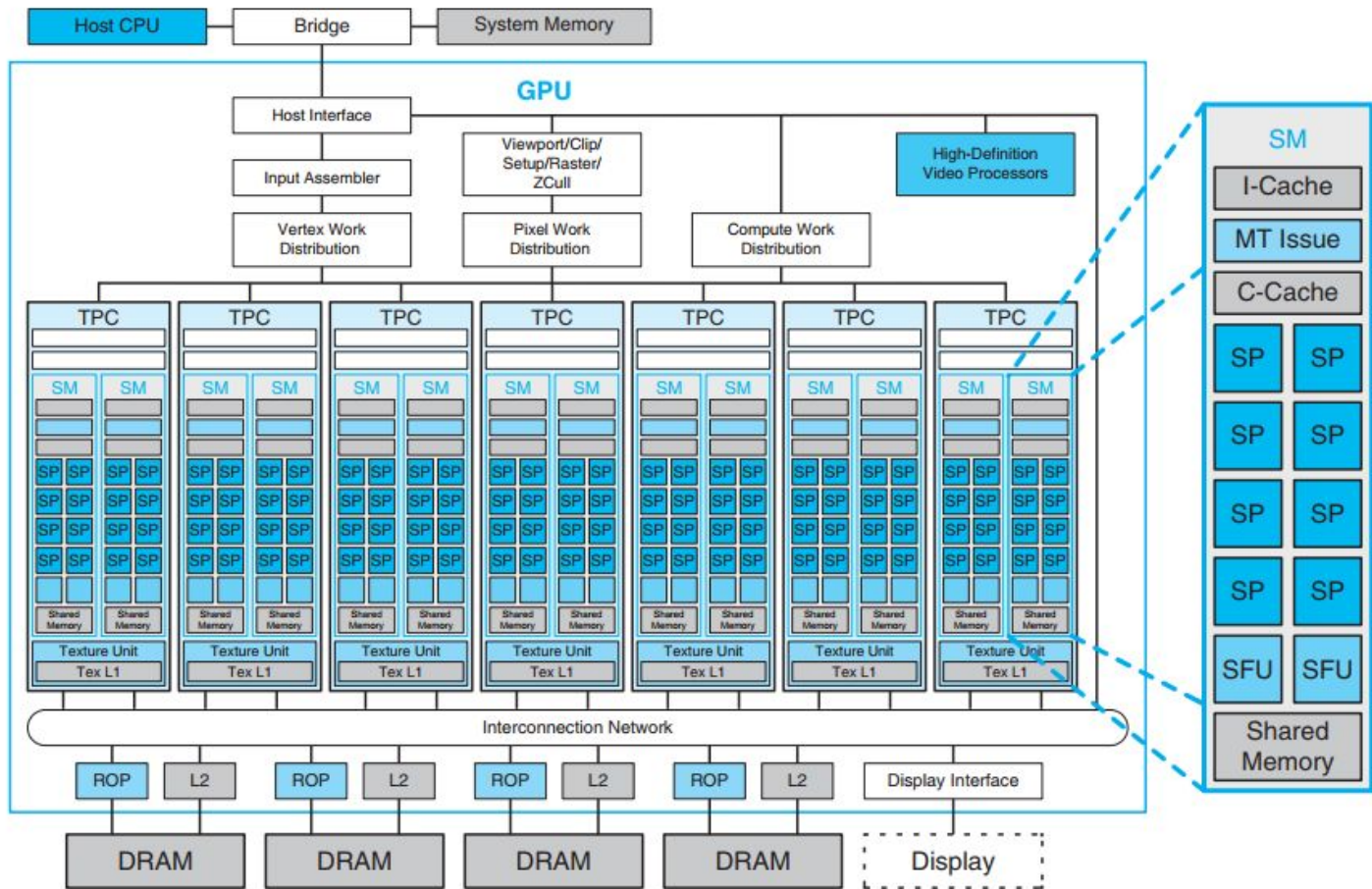
- **Unification:** Modern architectures unify vertex, geometry, pixel shader, and parallel computing on the same physical processors
- **Parallel Array:** Systems are based on a parallel array of many programmable processors rather than dedicated separate hardware for each shader type
- **Fixed-Function Integration:** The programmable array is tightly integrated with fixed-function processors for texture filtering, video decoding, anti-aliasing tasks



Physical Architecture: The Processor Array

NVIDIA GeForce 8800 (Tesla Architecture)

- **Streaming Multiprocessors (SMs):** The array is organized into multithreaded SMs
- **Core Density:** A typical configuration includes 112 Streaming Processor (SP) cores organized into 14 SMs
- **SM Components:** Each SM contains eight SP cores, two Special Function Units (SFUs), shared memory, and instruction/constant caches
- **Multithreading:** Each SP core is highly multithreaded, managing 96 concurrent threads and their hardware states



Basic unified GPU architecture

Architectural Design Philosophy: Throughput vs. Latency

- **Parallel Execution:** Design is focused on executing many parallel threads efficiently across many simpler cores
- **Computation Priority:** A larger portion of the chip's transistor budget is devoted to computation, with less devoted to on-chip caches and overhead compared to multicore CPUs
- **Scalability:** Processor arrays are designed to scale to different market segments by adjusting the number of multiprocessors and memory partitions

Introduction to GPU Programming: The Scalability Paradigm

- **GPU vs. CPU Parallelism:** GPUs provide two to three orders of magnitude more thread and data parallelism than CPUs
- **Hardware Scalability:** Architectures scale by adjusting the number of multiprocessors and memory partitions
- **Performance Growth:** Parallelism tends to double every 12 to 18 months, driven by Moore's Law and architectural efficiency
- **The Programmer's Goal:** Write code for a single thread; the GPU runs myriad instances in parallel, scaling transparently across different hardware tiers

The Real-Time Graphics Pipeline: Logical Dataflow Infrastructure

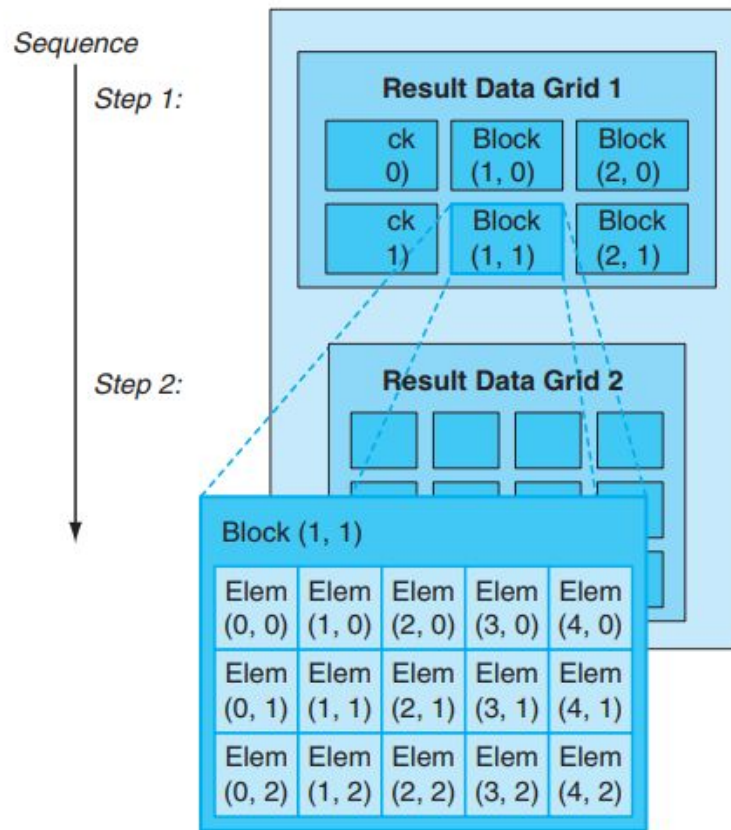
- **Input Assembler:** Collects vertices and groups them into primitives like triangles or polygons
- **Programmable Stages (The Shaders):**
 - **Vertex Shader:** Handles per-vertex processing, such as 3D-to-screen position transformation and lighting
 - **Geometry Shader:** Processes geometric primitives; can add/drop primitives
 - **Pixel Shader:** Performs per-fragment processing, including texturing and coloring
- **Fixed-Function Blocks:** Includes the Setup & Rasterizer (generates pixel fragments) and Output Merger (performs Z-buffer depth testing and color blending)

Beyond Graphics: The CUDA Model

- **What is CUDA?:** An extension of C/C++ developed by NVIDIA for scalable parallel programming
- **Problem Decomposition:** Large problems are partitioned into a grid of blocks, which are then broken down into individual elements
- **Three Key Abstractions:**
 - Hierarchy of thread groups
 - Shared memories
 - Barrier synchronization
- **The Kernel:** A C function for one sequential thread that executes in parallel across many threads

CUDA Thread Hierarchy: Managing Thousands of Concurrent Threads

- **Threads:** The finest unit of work; each has a unique ID (threadIdx)
- **Thread Blocks:** Concurrent threads that can cooperate via barrier synchronization and shared memory
- **Grids:** A set of thread blocks that execute the same kernel independently
- **Virtualization:** The number of blocks can be dictated by the data size rather than the physical processor count, allowing the same code to run on any GPU



Decomposing result data into a grid of blocks of elements to be computed in parallel.

Computing $y = ax + y$ with a serial loop:

```
void saxpy_serial(int n, float alpha, float *x, float *y)
{
    for(int i = 0; i<n; ++i)
        y[i] = alpha*x[i] + y[i];
}
// Invoke serial SAXPY kernel
saxpy_serial(n, 2.0, x, y);
```

Computing $y = ax + y$ in parallel using CUDA:

```
__global__
void saxpy_parallel(int n, float alpha, float *x, float *y)
{
    int i = blockIdx.x*blockDim.x + threadIdx.x;

    if( i<n ) y[i] = alpha*x[i] + y[i];
}

// Invoke parallel SAXPY kernel (256 threads per block)
int nblocks = (n + 255) / 256;
saxpy_parallel<<<nblocks, 256>>>(n, 2.0, x, y);
```

Sequential code in C versus parallel code in CUDA for SAXPY. CUDA parallel threads replace the C serial loop—each thread computes the same result as one loop iteration. The parallel code computes n results with n threads organized in blocks of 256 threads

```

#include <stdio.h>
#include <cuda_runtime.h>

#define N 1024

// CUDA kernel
__global__ void saxpy(float a, float *x, float *y) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < N) {
        y[i] = a * x[i] + y[i];
    }
}

int main() {
    float *x, *y, *d_x, *d_y;
    float a = 2.0f;

    // Allocate host memory
    x = (float*)malloc(N * sizeof(float));
    y = (float*)malloc(N * sizeof(float));

```

```

nvcc saxpy.cu -o saxpy
./saxpy

```

```

// Initialize host arrays
for (int i = 0; i < N; i++) {
    x[i] = 1.0f;
    y[i] = 2.0f;
}

// Allocate device memory
cudaMalloc((void**)&d_x, N * sizeof(float));
cudaMalloc((void**)&d_y, N * sizeof(float));

// Copy data from host to device
cudaMemcpy(d_x, x, N * sizeof(float), cudaMemcpyHostToDevice);
cudaMemcpy(d_y, y, N * sizeof(float), cudaMemcpyHostToDevice);

// Launch kernel
int blockSize = 256;
int numBlocks = (N + blockSize - 1) / blockSize;
saxpy<<<numBlocks, blockSize>>>(a, d_x, d_y);

// Copy result back to host
cudaMemcpy(y, d_y, N * sizeof(float), cudaMemcpyDeviceToHost);

// Verify result
for (int i = 0; i < 5; i++) {
    printf("y[%d] = %f\n", i, y[i]);
}

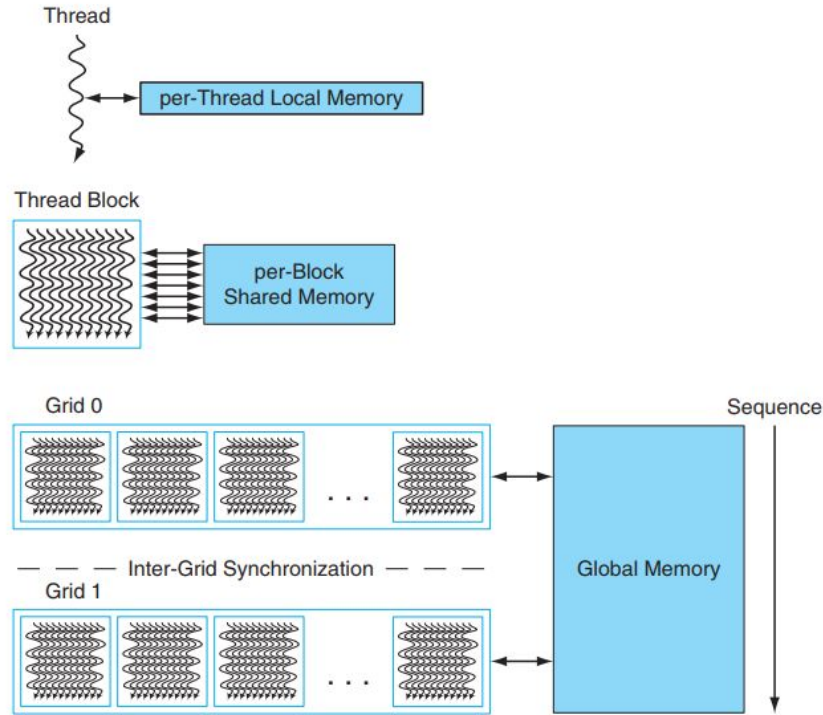
// Free memory
cudaFree(d_x);
cudaFree(d_y);
free(x);
free(y);

return 0;
}

```

CUDA Memory & Synchronization: Data Sharing and Constraints

- **Memory Levels:**
 - **Local Memory:** Private to each individual thread
 - **Shared Memory:** Fast, on-chip RAM shared by all threads within a single block
 - **Global Memory:** Accessible by all threads in an application; resides in the fast DRAM on the graphics board
- **Synchronization:** The `__syncthreads()` intrinsic acts as a barrier, ensuring all threads in a block reach a point before any proceed
- **Independence Rule:** Different blocks must be able to execute in any order (parallel or serial) to ensure scalability and avoid deadlocks

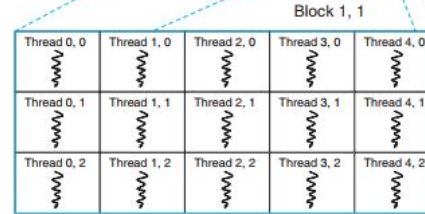
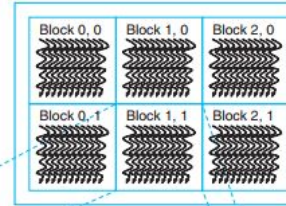


Nested granularity levels—thread, thread block, and grid—have corresponding memory sharing levels—local, shared, and global. Per-thread local memory is private to the thread. Per-block shared memory is shared by all threads of the block. Per-application global memory is shared by all threads.

Sequence

kernelF 2D Grid is 3×2 thread blocks; each block is 5×3 threads

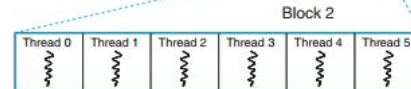
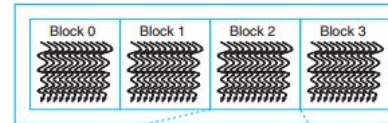
kernelF<<<(3, 2), (5, 3)>>>(params);



----- Interkernel Synchronization Barrier -----

kernelG 1D Grid is 4 thread blocks; each block is 6 threads

kernelG<<<4, 6>>>(params);



Sequence of kernel F instantiated on a 2D grid of 2D thread blocks, an interkernel synchronization barrier, followed by kernel G on a 1D grid of 1D thread blocks

Architectural Summary: What Drives GPU Design?

- **Fine-Grained Data Parallelism:** Describing how to process one pixel/vertex or compute one result
- **High Throughput:** Creating and executing millions of threads per frame at 60 frames per second
- **Transparent Scalability:** Increasing performance automatically with more processors—no recompilation required
- **Floating-Point Intensity:** Heavy reliance on floating-point arithmetic for complex lighting and dynamic range